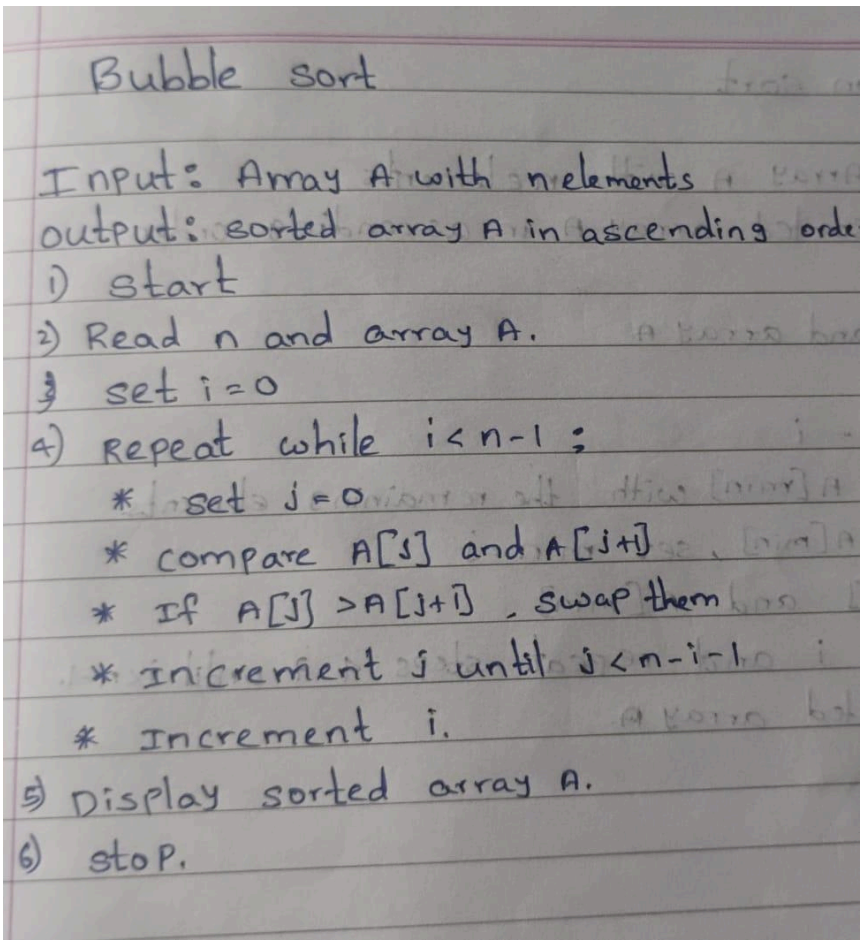
# PRACTICAL - 1

**AIM : Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort**

BUBBLE SORT ALGORITHM




PROGRAM:

```
#include <iostream>
using namespace std;
class bubblesort
{
public:
   int size, arr[100], i;
   void arrayinput()
```

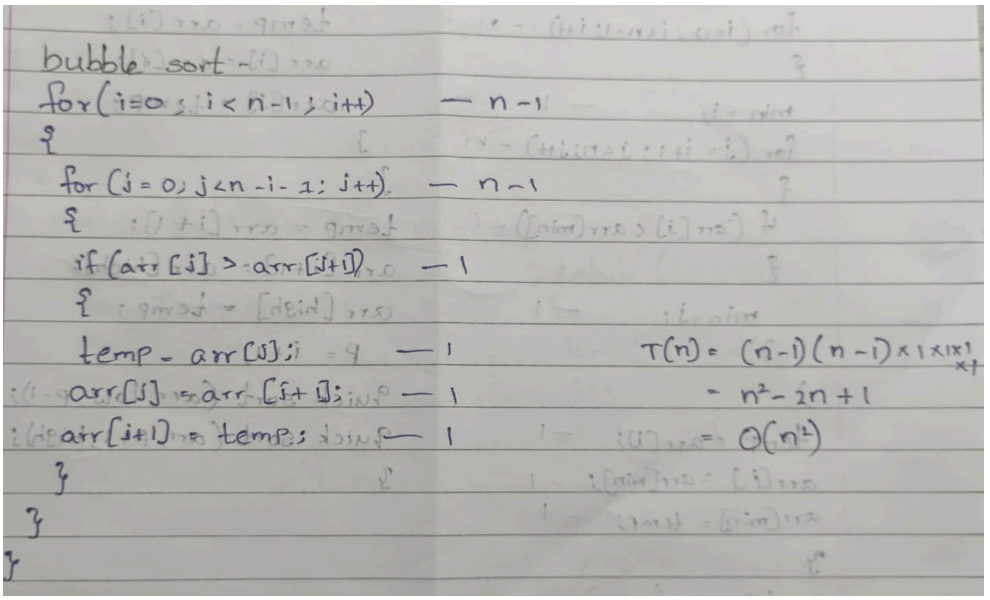
```
    {
        cout << "Enter array size: ";
        cin >> size;
        cout << "Enter array elements:\n";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }
    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
    void bubblelogic()
    {
        int j, temp;
        for (i = 0; i < size - 1; i++)
        {
            for (j = 0; j < size - i - 1; j++)
            {
                if (arr[j] > arr[j + 1])
                {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
};
int main()
{
    bubblesort bs;
    bs.arrayinput();
    cout << "Before sorting: ";
    bs.printarray();
    bs.bubblelogic();
    cout << "After sorting: ";
```

```
    bs.printarray();
    return 0;
}
```

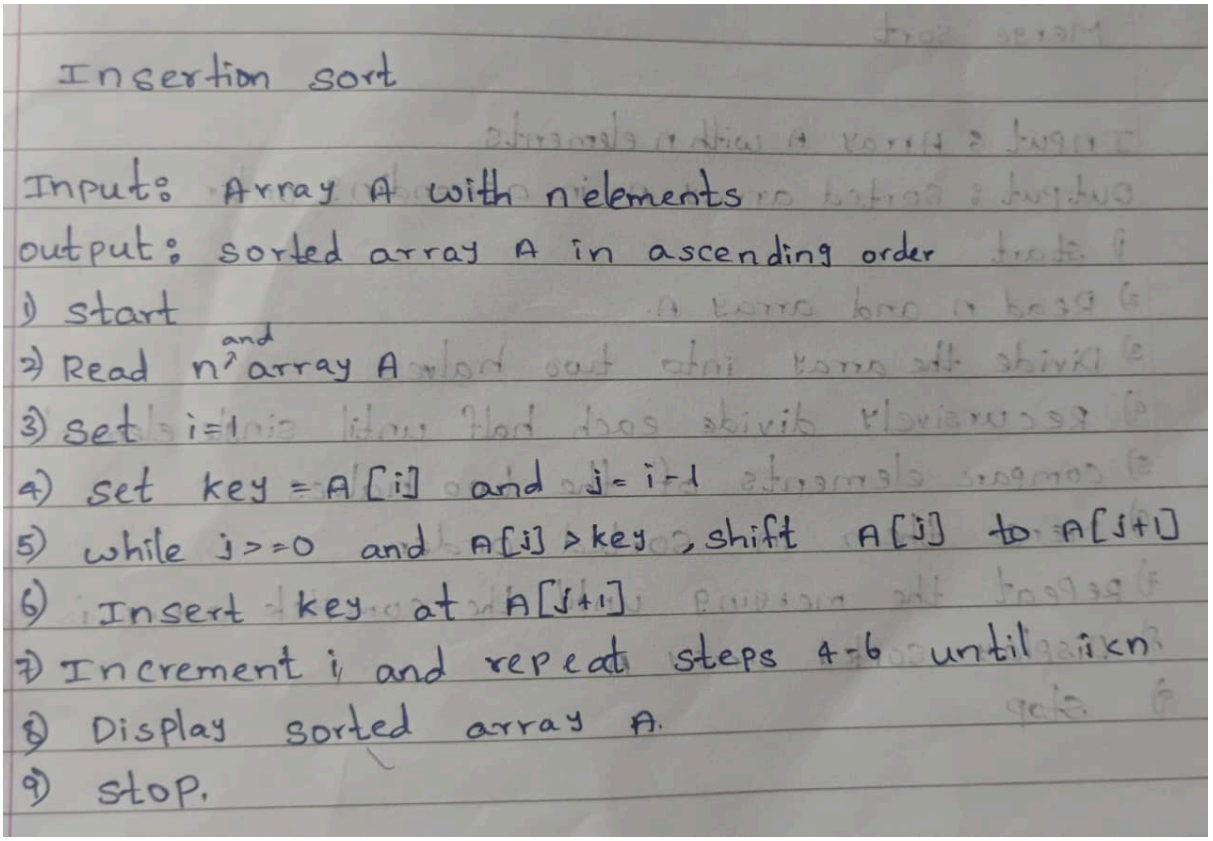
Output

```
Enter array size: 4
Enter array elements:
22
55
43
23
Before sorting: 22 55 43 23
After sorting: 22 23 43 55
```

TIME COMPLEXITY : Step Count / Frequency Count Method )



INSERTION ALGORITHM




PROGRAM:

```
#include <iostream>
using namespace std;

class InsertionSort
{
public:
   int size, arr[100], i, j, temp;

   void arrayinput()
   {
      cout << "Enter array size: ";
      cin >> size;
```

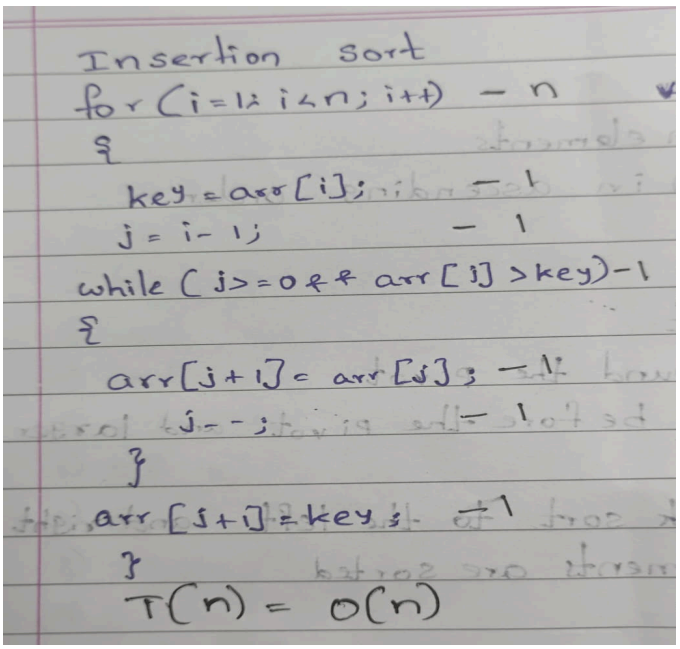
```
        cout << "Enter array elements: ";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void insertionsortlogic()
    {
        for (i = 1; i < size; i++)
        {
            temp = arr[i];
            j = i - 1;

            while (j >= 0 && arr[j] > temp)
            {
                arr[j + 1] = arr[j];
                j--;
            }

            arr[j + 1] = temp;
        }
    }
};

int main()
{
    InsertionSort is;

    is.arrayinput();

    cout << "Before sorting: ";
    is.printarray();
```

```
    is.insertionsortlogic();

    cout << "After sorting: ";
    is.printarray();

    return 0;
}
```

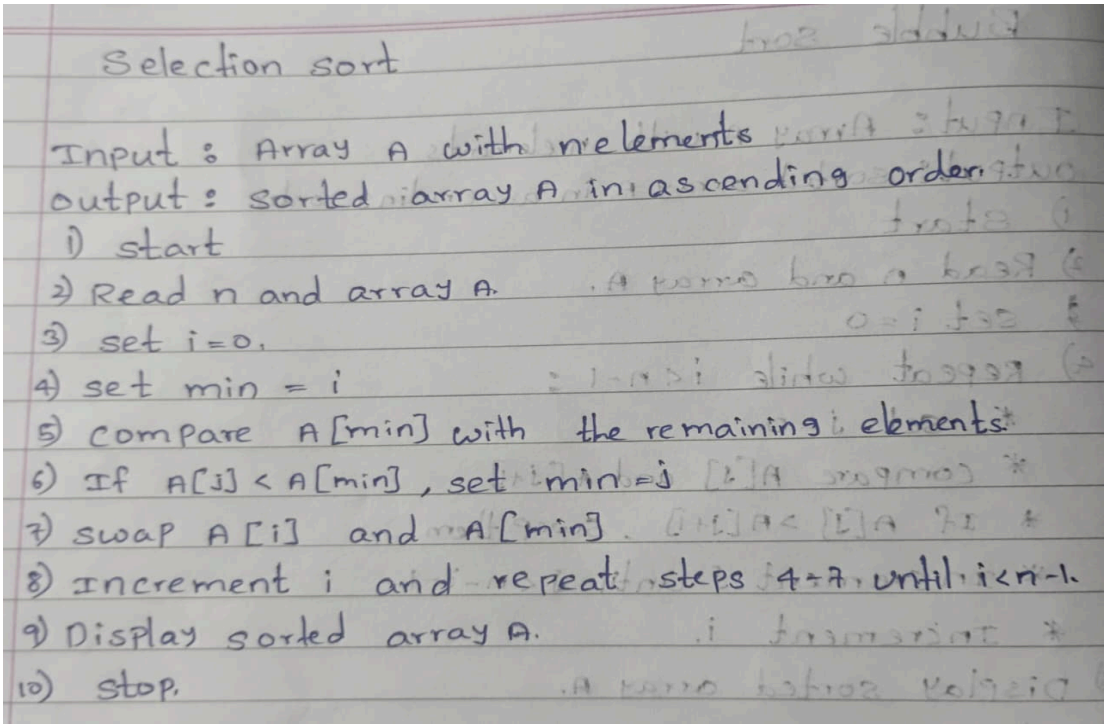
Output

```
Enter array size: 4
Enter array elements: 12
34
76
1
Before sorting: 12 34 76 1
After sorting: 1 12 34 76
```

TIME COMPLEXITY : Step Count / Frequency Count Method )



SELEECTION SORT ALGORITHM




PROGRAM:

```
#include <iostream>
using namespace std;

class SelectionSort
{
public:
   int size, arr[100], i, j, temp;

   void arrayinput()
   {
      cout << "Enter array size: ";
      cin >> size;

      cout << "Enter array elements: ";
      for (i = 0; i < size; i++)
      {
         cin >> arr[i];
      }
   }
```

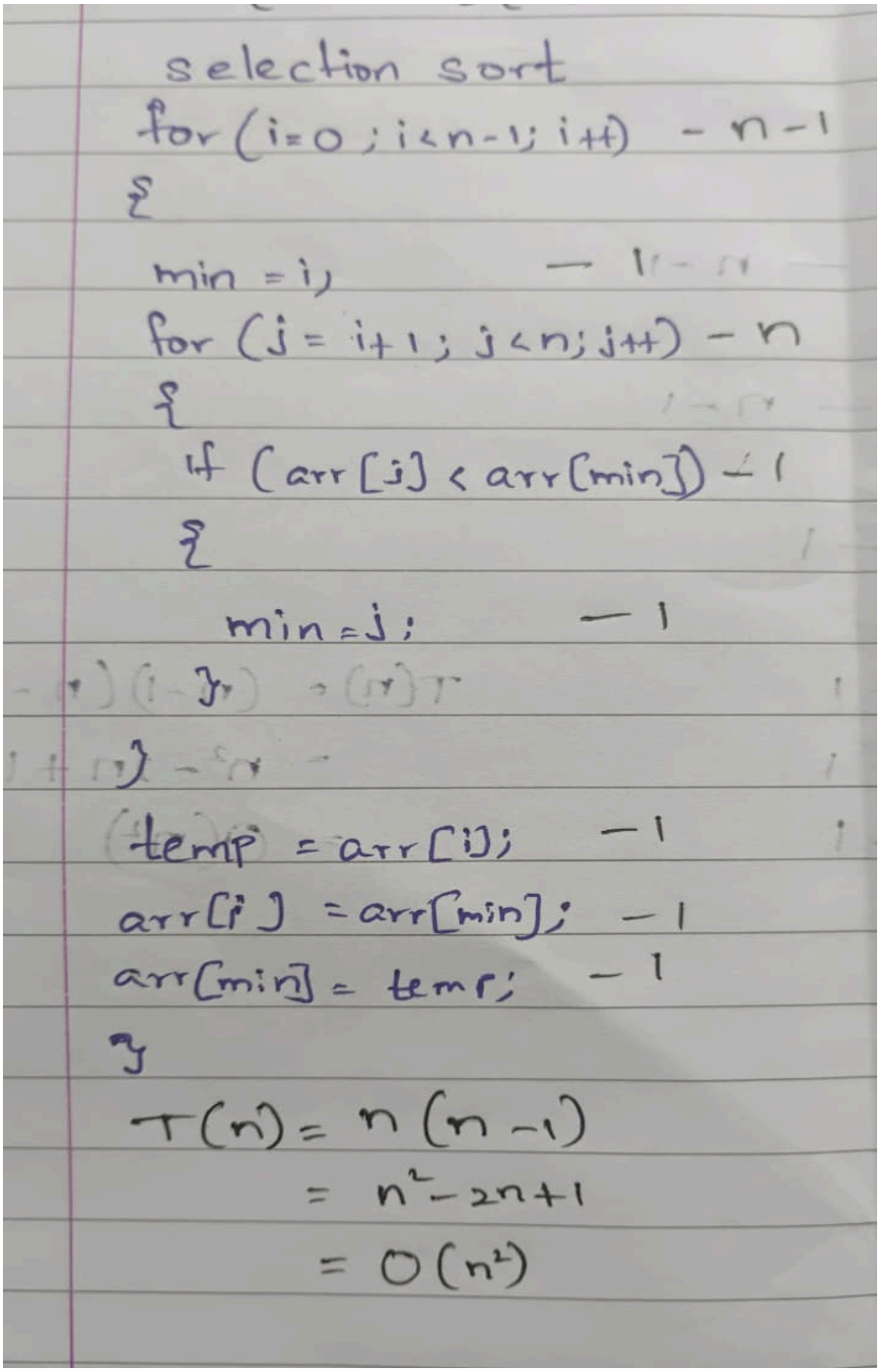
```
    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void selectionsortlogic()
    {
        for (i = 0; i < size - 1; i++)
        {
            int min = i;

            for (j = i + 1; j < size; j++)
            {
                if (arr[j] < arr[min])
                {
                    min = j;
                }
            }

            if (min != i)
            {
                temp = arr[i];
                arr[i] = arr[min];
                arr[min] = temp;
            }
        }
    }
};

int main()
{
    SelectionSort ss;

    ss.arrayinput();

    cout << "Before sorting: ";
    ss.printarray();
```

```
    ss.selectionsortlogic();

    cout << "After sorting: ";
    ss.printarray();

    return 0;
}
```

Output

```
Enter array size: 4
Enter array elements: 23
46
11
98
Before sorting: 23 46 11 98
After sorting: 11 23 46 98
```

time complexity : Step Count / Frequency Count Method )



selection sort

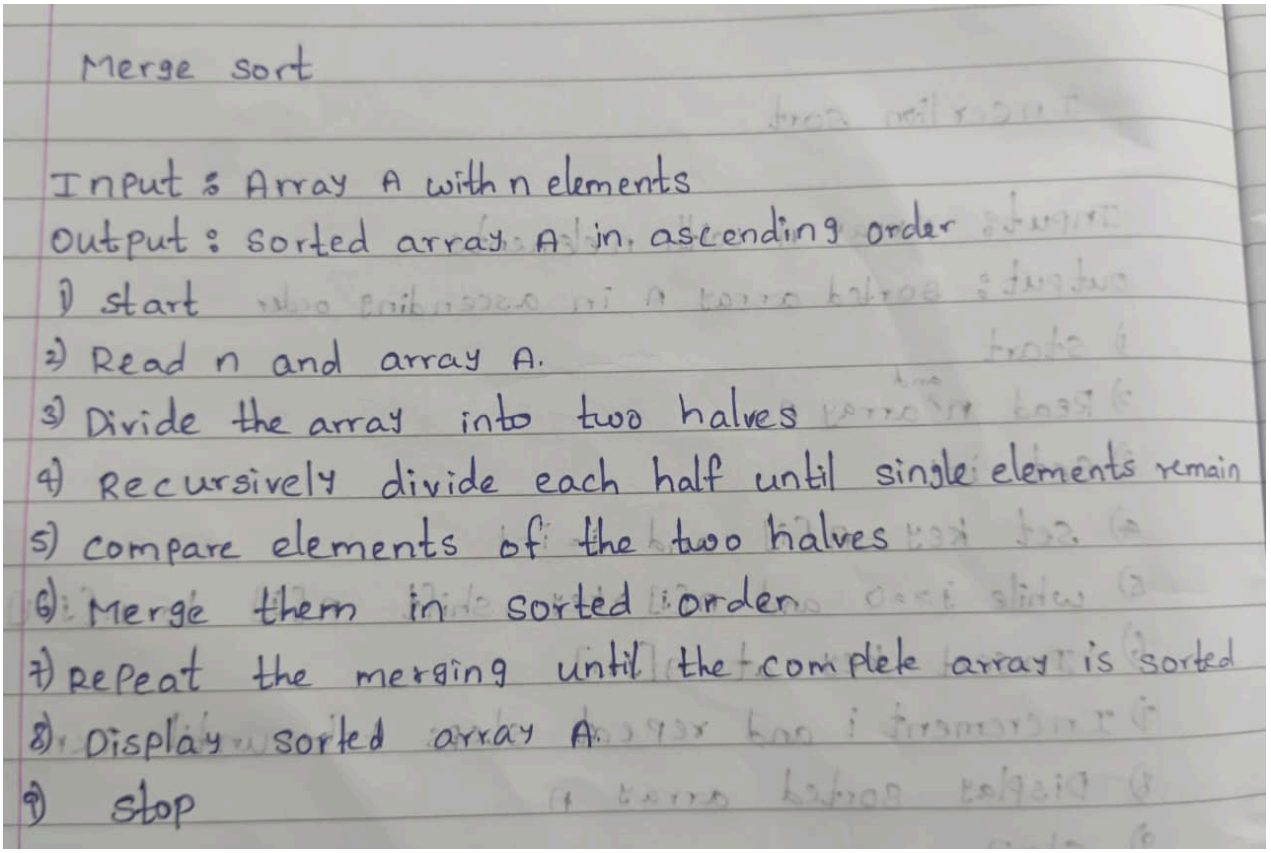
```
for (i=0; i<n-1; i++)   - n-1
{
 min = i;               - 1
 for (j = i+1; j<n; j++) - n
 {
  if (arr[j] < arr[min]) - 1
  {
    min = j;            - 1
  }
 }
 temp = arr[i];         - 1
 arr[i] = arr[min];     - 1
 arr[min] = temp;       - 1
}
```

$T(n) = n(n-1)$

$= n^2 - 2n + 1$

$= O(n^2)$

MERGE SORT ALGORITHM :




PROGRAM:

```
#include <iostream>
using namespace std;

class mergesort
{
public:
   int size, arr[100], i;

   void arrayinput()
   {
      cout << "Enter array size: ";
      cin >> size;

      cout << "Enter array elements:\n";
      for (i = 0; i < size; i++)
```

```
    {
        cin >> arr[i];
    }
}

void printarray()
{
    for (i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void merge(int low, int mid, int high)
{
    int temp[100];
    int i = low, j = mid + 1, k = low;

    while (i <= mid && j <= high)
    {
        if (arr[i] <= arr[j])
        {
            temp[k] = arr[i];
            i++;
        }
        else
        {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= mid)
    {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= high)
```

```
        {
            temp[k] = arr[j];
            j++;
            k++;
        }

        for (i = low; i <= high; i++)
        {
            arr[i] = temp[i];
        }
    }

    void mergelogic(int low, int high)
    {
        if (low < high)
        {
            int mid = (low + high) / 2;

            mergelogic(low, mid);
            mergelogic(mid + 1, high);

            merge(low, mid, high);
        }
    }
};

int main()
{
    mergesort ms;

    ms.arrayinput();

    cout << "Before sorting: ";
    ms.printarray();

    ms.mergelogic(0, ms.size - 1);

    cout << "After sorting: ";
    ms.printarray();

    return 0;
}
```

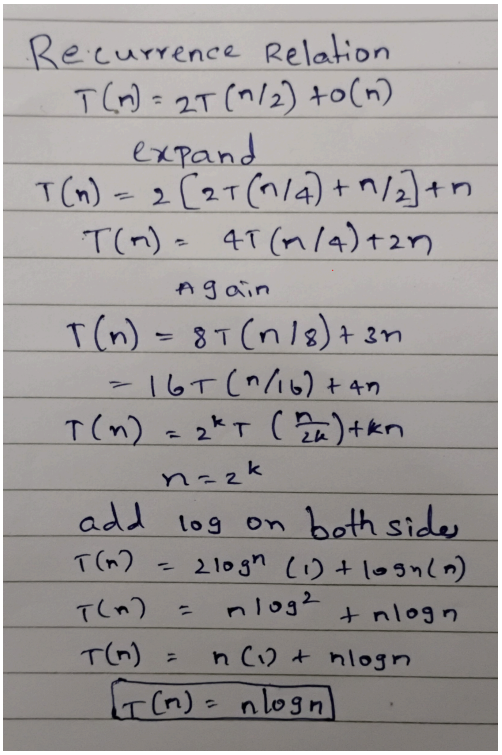
Output

```
Enter array size: 4
Enter array elements:
12
45
23
11
Before sorting: 12 45 23 11
After sorting: 11 12 23 45
```

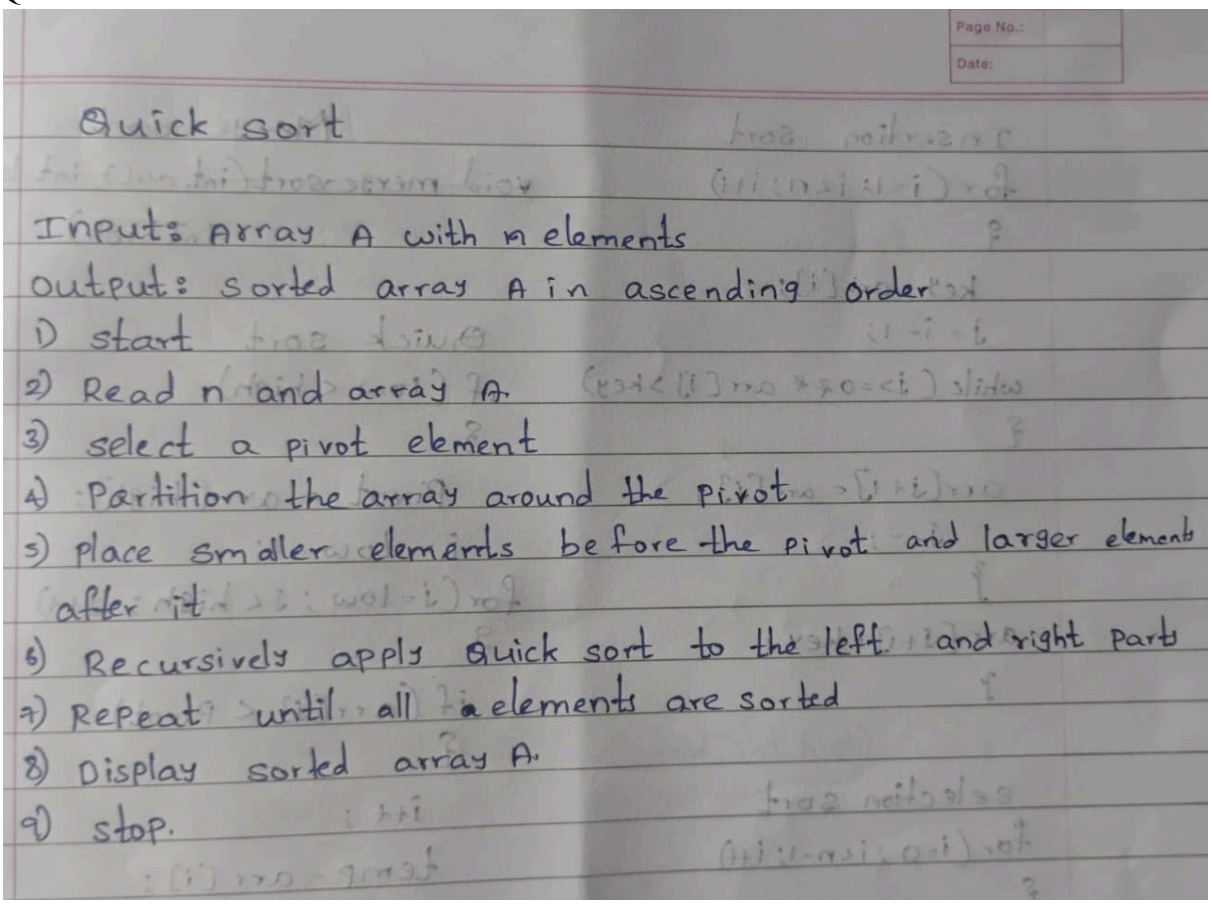
time complexity : Step Count / Frequency Count Method )

Merge sort

```
void mergesort (int arr[], int low, int high)
{
 if (low < high)                          ——— 1
 {
  mid = (low + high) / 2;                 ——— 1
 mergesort (arr, low, mid);               ——— T(n/2)
 merge sort (arr, mid+1, high);           ——— T(n/2)
 merge (arr, low, mid, high);             ——— O(n)
 }
}
void merge (int arr[], int low, int mid, int high)
 {
   i = low;                               ——— O(1)
   j = mid + 1;                           ——— 1
   k = low;                               ——— (1)
while (i <= mid && j <= high)             ——— (n)
{
 if( arr[i] < arr[j])                     ——— 1;
 {
  temp[k] = arr[i];                       ——— 1
   i++
 }
 else
{
 temp[k] = arr[j];                        ——— 1
   j++;
 }
 k++;
}
while (i <= mid)                          ——— O(n)
{
 temp[k] = arr[i];                        ——— 1
 i++;                                     ——— 1
 k++;                                     ——— 1
}
while (j <= high) {                       ——— O(n)
 temp[k] = arr[i];                        ——— 1
   j++;                                   ——— 1
   k++; }                                 ——— 1
for (i = low; i <= high; i++)
{ arr[i] = temp[i];
```


Recurrence Relation
T(n) = 2T(n/2) + o(n)
expand
T(n) = 2[2T(n/4) + n/2] + n
T(n) = 4T(n/4) + 2n
Again
T(n) = 8T(n/8) + 3n
= 16T(n/16) + 4n
T(n) = 2^k T(n/2^k) + kn
n = 2^k
add log on both side
T(n) = 2log^n (1) + log n(n)
T(n) = n log^2 + n log n
T(n) = n(1) + n log n
T(n) = n log n

QUICK SORT ALGORITHM:




PROGRAM :

```
#include <iostream>
#include <utility>
using namespace std;

class QuickSort
{
public:
   int size, arr[100];

   void arrayinput()
   {
      cout << "Enter array size: ";
      cin >> size;

      cout << "Enter array elements: ";
```

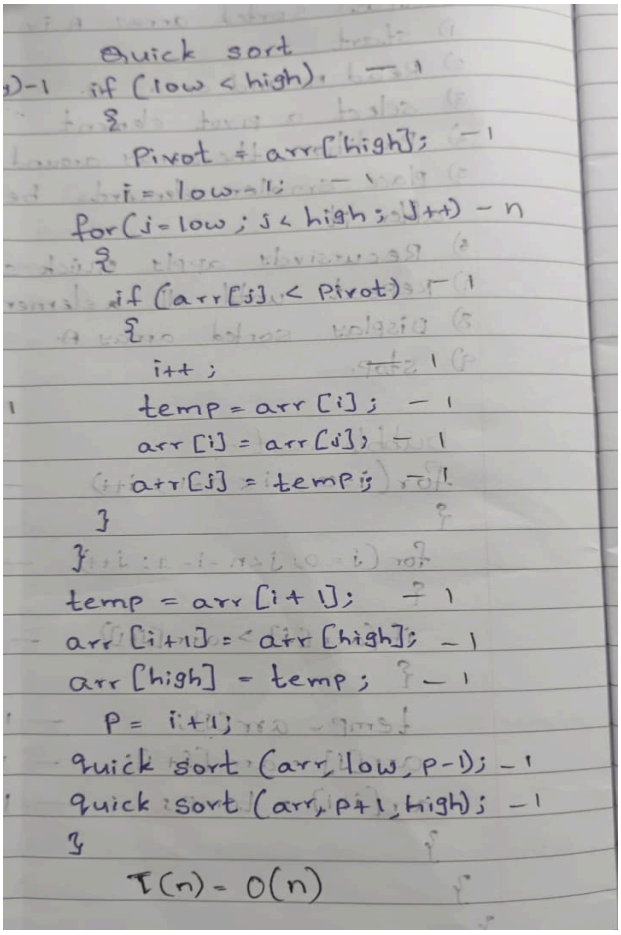
```
    for (int i = 0; i < size; i++)
    {
        cin >> arr[i];
    }
}

void printarray()
{
    for (int i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int partition(int lb, int ub)
{
    int pivot = arr[lb];
    int start = lb;
    int end = ub;

    while (start < end)
    {
        while (start <= ub && arr[start] <= pivot)
            start++;

        while (arr[end] > pivot)
            end--;

        if (start < end)
            swap(arr[start], arr[end]);
    }

    swap(arr[lb], arr[end]);
    return end;
}

void quicksortlogic(int lb, int ub)
{
    if (lb < ub)
    {
        int loc = partition(lb, ub);
```

```
            quicksortlogic(lb, loc - 1);
            quicksortlogic(loc + 1, ub);
        }
    }

    void sort()
    {
        quicksortlogic(0, size - 1);
    }
};

int main()
{
    QuickSort qs;

    qs.arrayinput();

    cout << "Before sorting: ";
    qs.printarray();

    qs.sort();

    cout << "After sorting: ";
    qs.printarray();

    return 0;
}
```

Output

```
Enter array size: 4
Enter array elements: 23
43
55
12
Before sorting: 23 43 55 12
After sorting: 12 23 43 55
```

TIME COMPLEXITY : Step Count / Frequency Count Method )



```
Quick sort
if (low < high)
{
  Pivot = arr[high];
  i = low - 1;
  for(j = low; j < high; j++)
  {
    if (arr[j] < pivot)
    {
      i++;
      temp = arr[i];
      arr[i] = arr[j];
      arr[j] = temp;
    }
  }
  temp = arr[i+1];
  arr[i+1] = arr[high];
  arr[high] = temp;
  p = i+1;
  quick sort(arr, low, p-1);
  quick sort(arr, p+1, high);
}
T(n) = O(n)
```